

IOT PROTOTYPE WITH ARDUINO / RASPBERRY PI

SMART DOOR LOCK SYSTEM

1. Introduction

The Smart Door Lock System is a simple IoT-based security project that allows users to lock and unlock a door using a password, RFID card, or mobile application. The project uses an Arduino Uno or Raspberry Pi connected with sensors and actuators to provide secure access control.

This project demonstrates the integration of IoT technology with embedded systems to create a smart home security solution.

2. Objectives of the Project

- To develop a smart IoT-based door locking system
- To improve home and office security
- To control door access remotely
- To monitor door status using IoT
- To reduce unauthorized access
- To demonstrate IoT communication using Arduino/Raspberry Pi

3. Components Required

Component	Quantity
Arduino Uno / Raspberry Pi	1
Servo Motor / Solenoid Lock	1
Keypad (4x4)	1
RFID Module (Optional)	1
ESP8266 Wi-Fi Module	1
LCD Display	1
Breadboard	1
Jumper Wires	Several
Power Supply	1

4. Working Principle

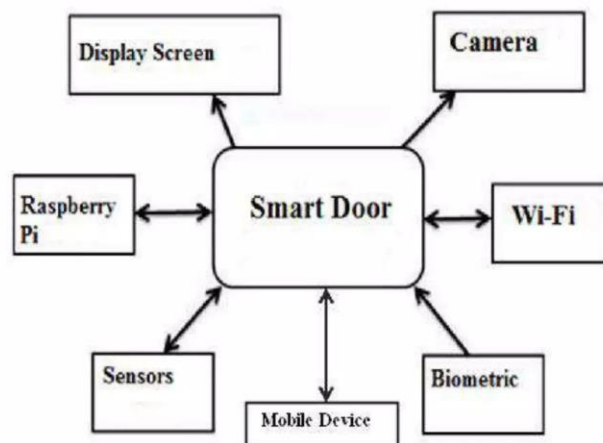
The Smart Door Lock System works using a password authentication mechanism.

Step-by-Step Working:

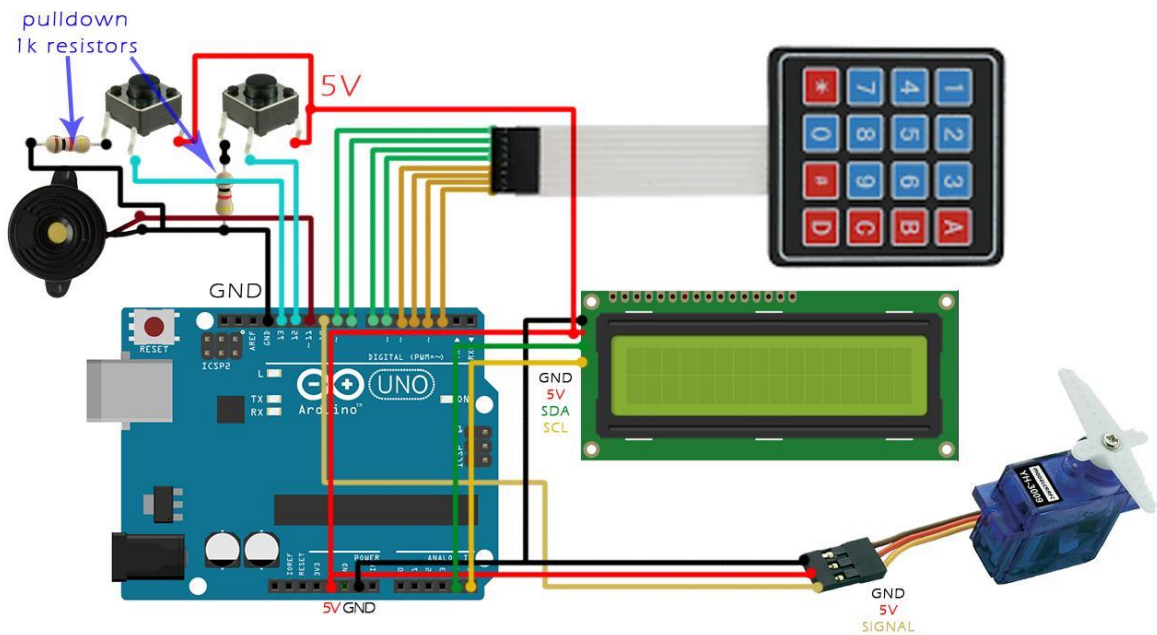
1. User enters password using keypad.
2. Arduino checks the entered password.
3. If the password is correct:
 - Servo motor rotates.
 - Door unlocks.
4. If the password is incorrect:
 - Access is denied.
 - Alarm or warning message is displayed.
5. ESP8266 sends door status to cloud/mobile app.
6. User can monitor the system remotely.

5. System Architecture

SYSTEM ARCHITECTURE



6. Circuit Diagram



7. Arduino Code

```
#include <Keypad.h>
```

```
#include <Servo.h>
```

```
Servo myservo;
```

```
const byte ROWS = 4;
```

```
const byte COLS = 4;
```

```
char keys[ROWS][COLS] = {  
  {'1','2','3','A'},  
  {'4','5','6','B'},  
  {'7','8','9','C'},  
  {'*','0','#','D'}  
};
```

```
byte rowPins[ROWS] = {9,8,7,6};
```

```
byte colPins[COLS] = {5,4,3,2};
```

```
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
```

```
String password = "1234";
```

```
String input = "";
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  myservo.attach(10);
```

```
  myservo.write(0);
```

```
  Serial.println("Smart Door Lock System");
```

```
}
```

```
void loop() {
```

```
  char key = keypad.getKey();
```

```
  if (key) {
```

```
    Serial.println(key);
```

```
    if (key == '#') {
```

```
      if (input == password) {
```

```
        Serial.println("Access Granted");
```

```
        myservo.write(90);
```

```
        delay(5000);
```

```
        myservo.write(0);
```

```
      }
```

```
    else {
```

```
      Serial.println("Wrong Password");
```

```
    }
```

```
    input = "";
```

```

}
else {
input += key;
}
}
}

```

8. Raspberry Pi Python Code (Optional)

```

import RPi.GPIO as GPIO
from time import sleep

servo = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(servo, GPIO.OUT)

pwm = GPIO.PWM(servo, 50)
pwm.start(0)

while True:
password = input("Enter Password: ")

if password == "1234":
print("Door Unlocked")
pwm.ChangeDutyCycle(7)
sleep(5)
pwm.ChangeDutyCycle(2)
else:
print("Wrong Password")

```

9. Output Images

Figure 1: Smart Door Lock Prototype

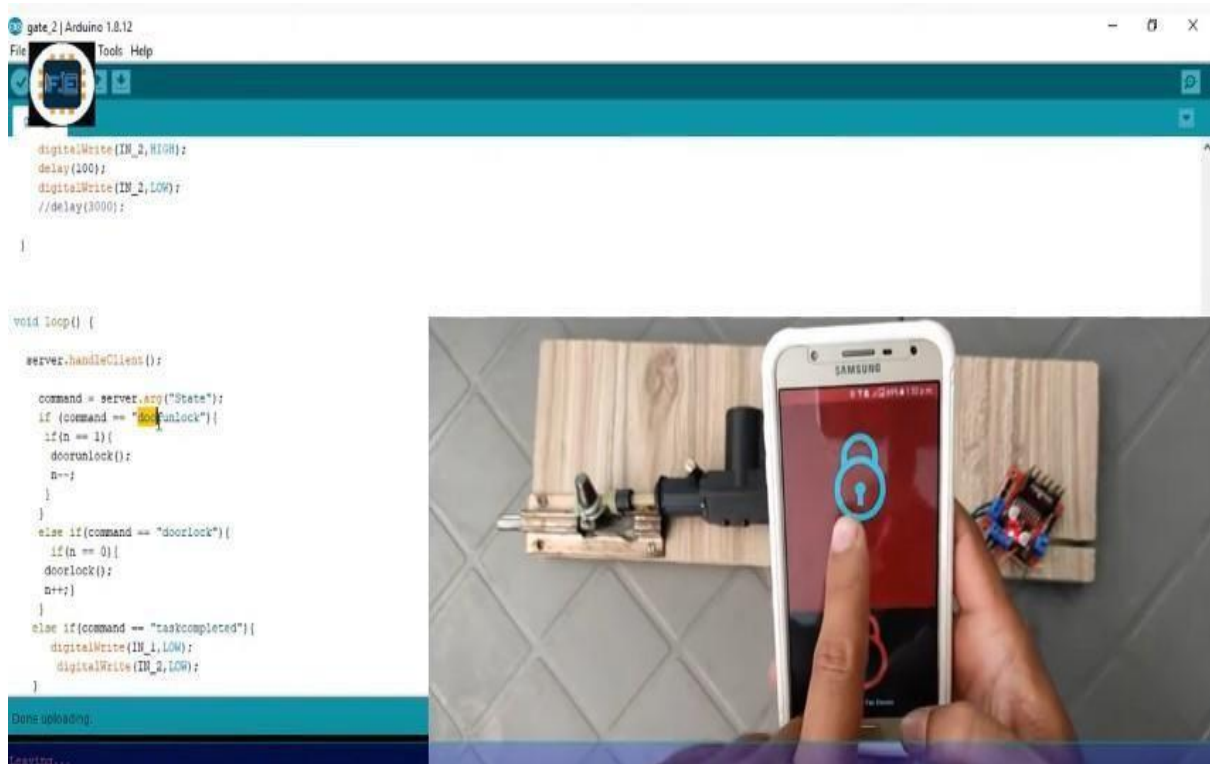


Figure 2: Password Authentication Display

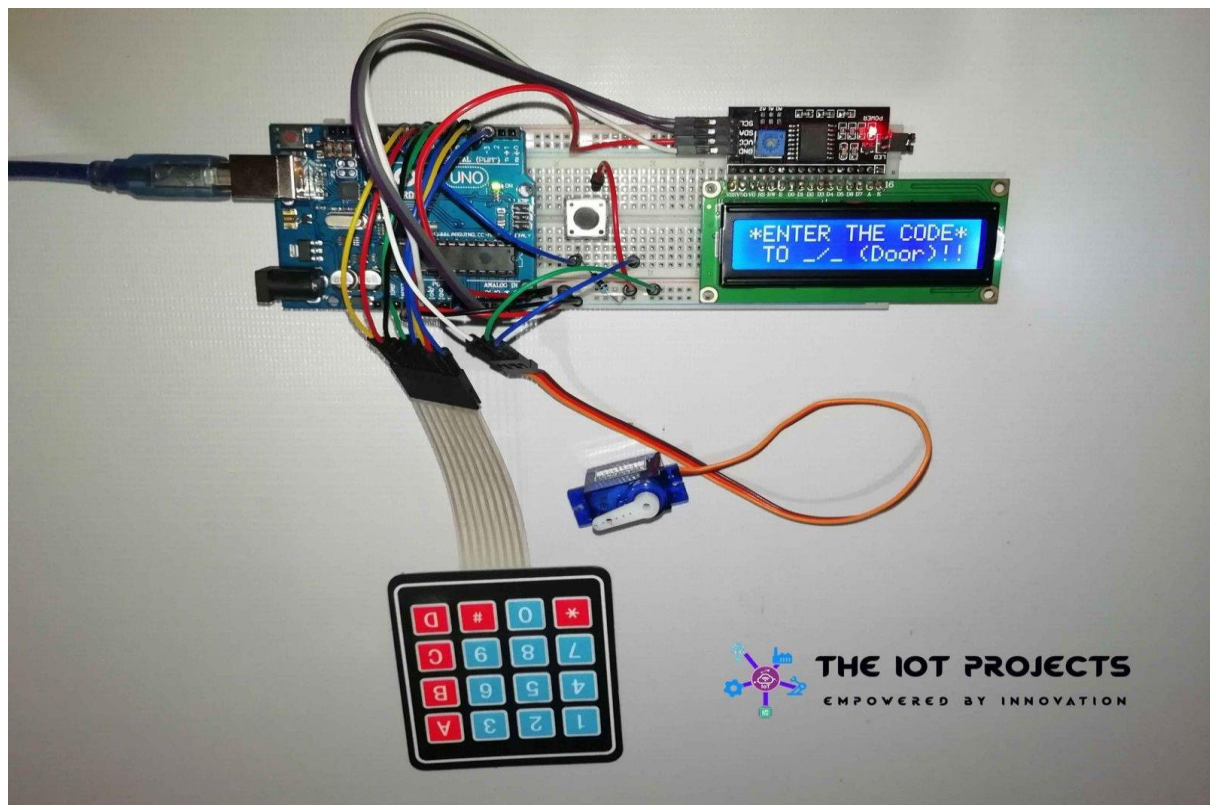


Figure3: Mobile Monitoring Application



10. Advantages

- Improved security
- Easy remote monitoring
- Low cost implementation
- Simple automation
- Real-time access control
- Suitable for smart homes

11. Applications

- Smart homes
- Offices
- Banks
- Laboratories
- Hotel rooms
- Industrial security systems

12. Future Enhancements

- Face recognition integration
- Fingerprint authentication
- Mobile app control
- Cloud database storage
- Voice assistant integration
- AI-based intrusion detection

13. Conclusion

The Smart Door Lock System is an effective IoT-based mini project that improves security and automation. Using Arduino or Raspberry Pi, the system provides password-based access control and remote monitoring features.

The project demonstrates how IoT technology can be applied in real-life smart home applications. It is simple, cost-effective, and suitable for students learning IoT and embedded systems.

TEMPERATURE & HUMIDITY MONITORING SYSTEM

1. Introduction

The Temperature & Humidity Monitoring System is a simple IoT project that measures environmental temperature and humidity using a DHT11/DHT22 sensor. The collected data is displayed on an LCD screen and can also be monitored remotely using Wi-Fi and cloud platforms.

This project is useful in smart homes, agriculture, industries, weather stations, and server rooms.

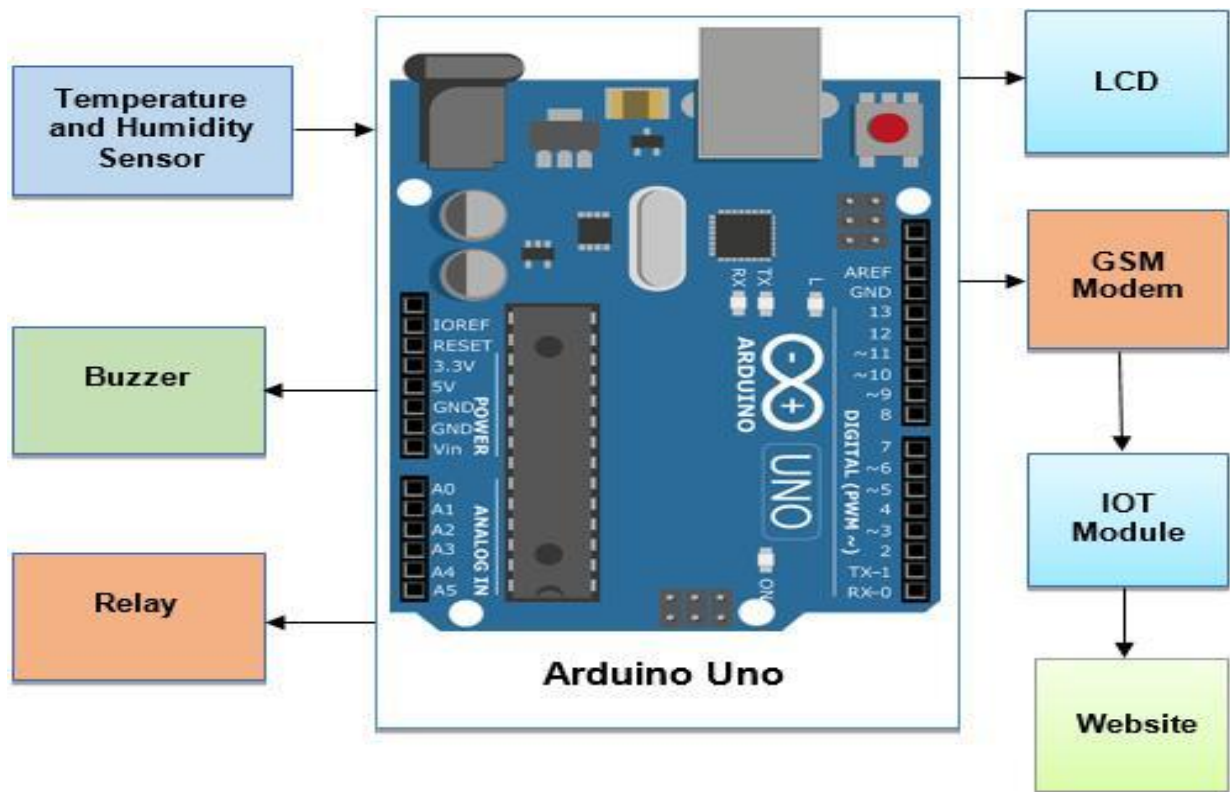
2. Components Required

Component	Quantity
Arduino Uno / Raspberry Pi	1
DHT11 or DHT22 Sensor	1
ESP8266 Wi-Fi Module	1
LCD Display (16x2)	1
Breadboard	1
Jumper Wires	Several
USB Cable / Power Supply	1

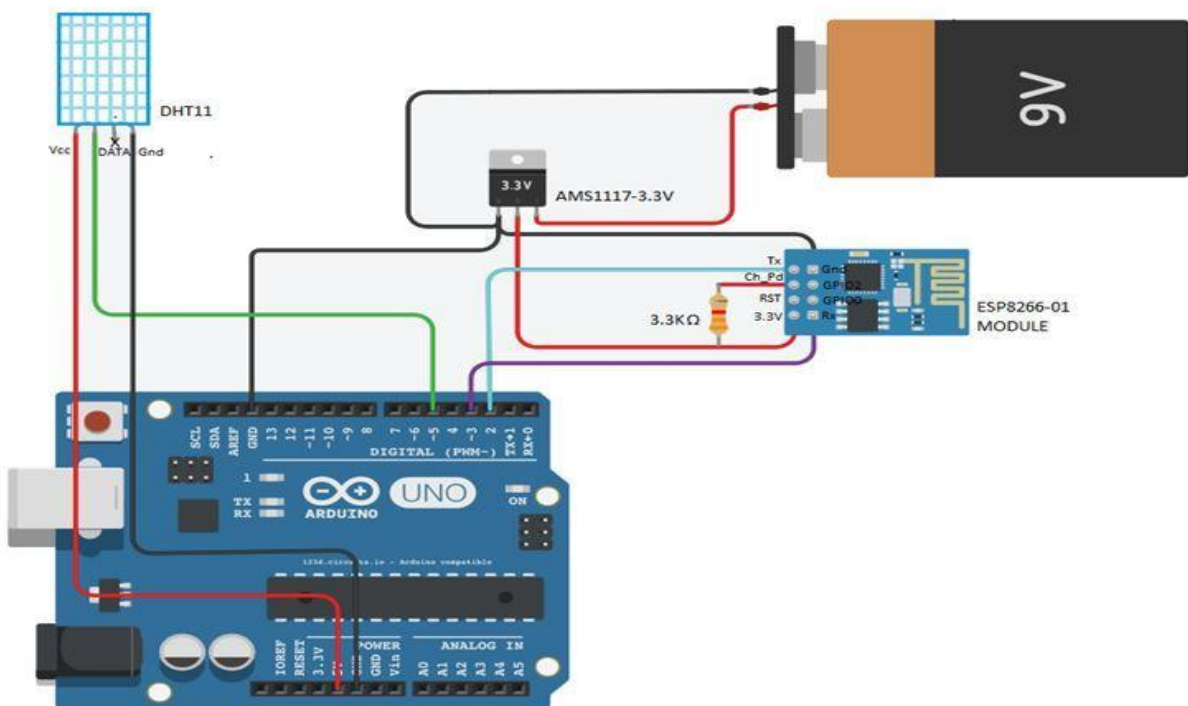
3. Working Principle

1. DHT11 sensor measures temperature and humidity.
2. Arduino/Raspberry Pi reads sensor values.
3. Data is displayed on LCD screen.
4. ESP8266 sends data to cloud/mobile app.
5. Users monitor readings remotely through IoT dashboard.

4. System Architecture Figure



5. Circuit Diagram Figure



6.Arduino Code

```
#include <DHT.h>

#define DHTPIN 2      // DHT11 connected to Digital Pin 2
#define DHTTYPE DHT11 // Sensor Type

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();

  Serial.println("Temperature & Humidity Monitoring System");
}

void loop() {
  // Read Humidity
  float humidity = dht.readHumidity();

  // Read Temperature in Celsius
  float temperature = dht.readTemperature();

  // Check if sensor reading failed
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  // Print Temperature
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  // Print Humidity
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");
```

```

Serial.println("-----");
delay(2000); // Wait 2 seconds
}

```

7. Raspberry Pi Python Code

```

import Adafruit_DHT
import time

sensor = Adafruit_DHT.DHT11
pin = 4
print("Temperature & Humidity Monitoring System")
while True:

    humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
    if humidity is not None and temperature is not None:

        print("Temperature = {:.1f} °C".format(temperature))
        print("Humidity = {:.1f} %".format(humidity))
        print("-----")

    else:

        print("Sensor Failure. Check Wiring.")

    time.sleep(2)

```

8. Output Images

Figure 1: Hardware Prototype

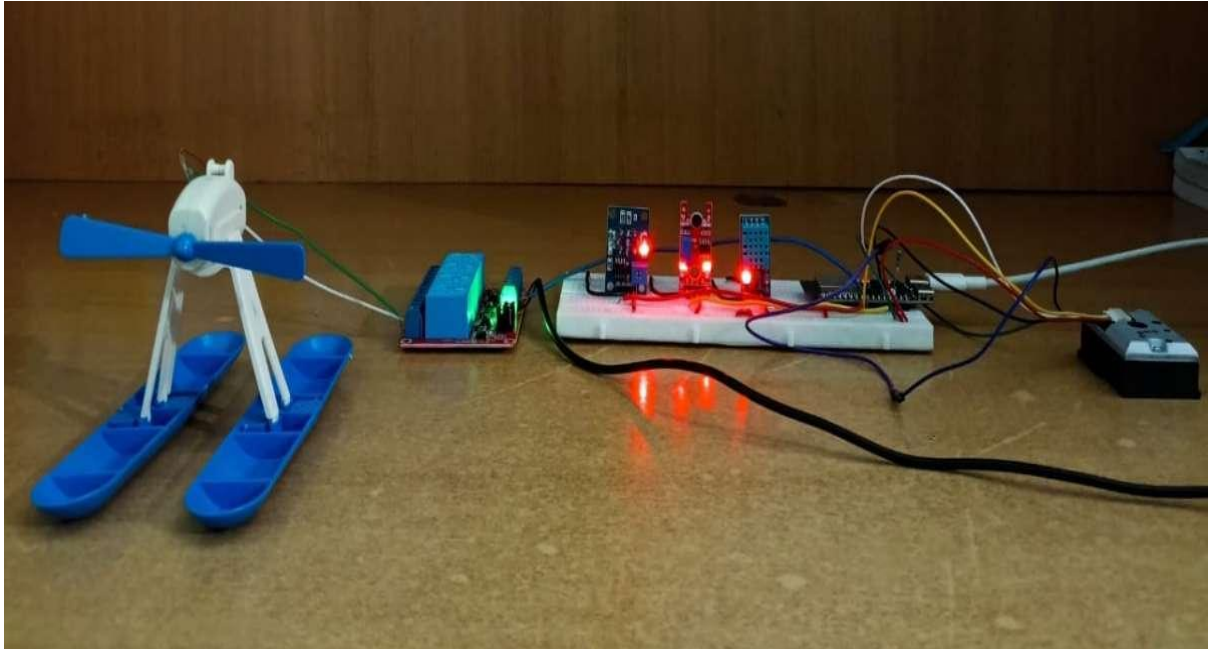


Figure 2: LCD Output Display

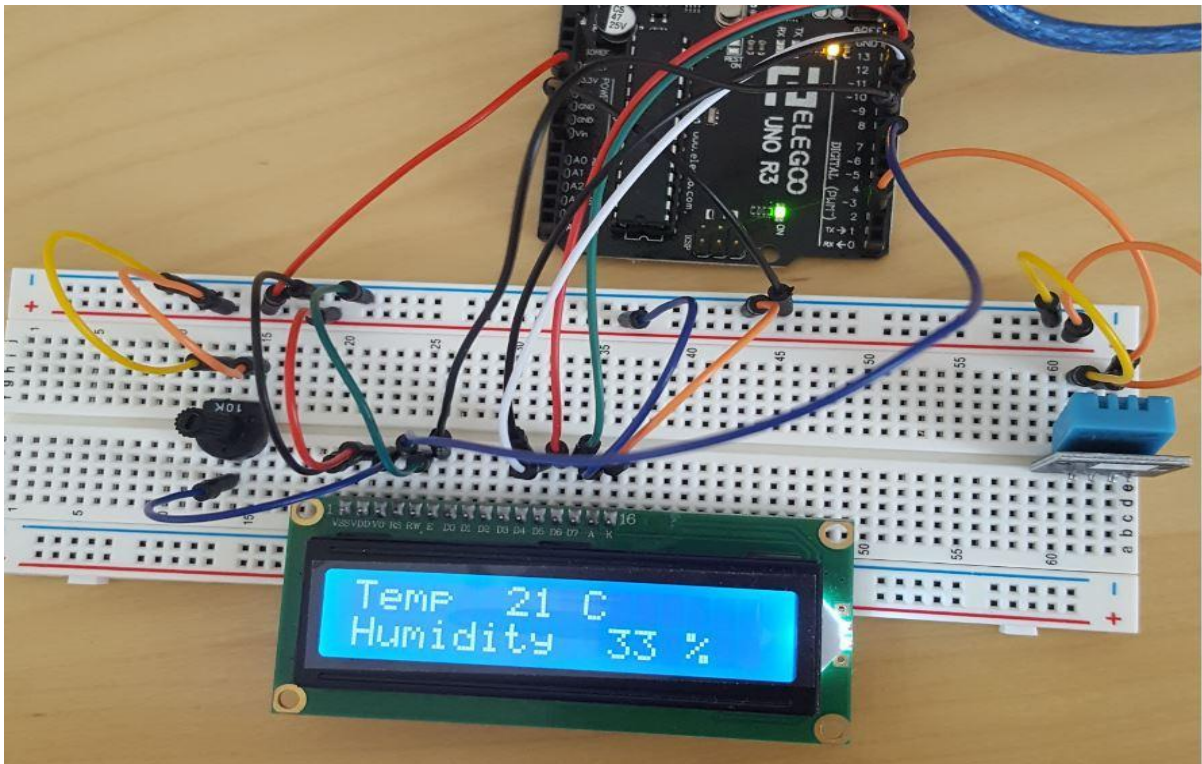
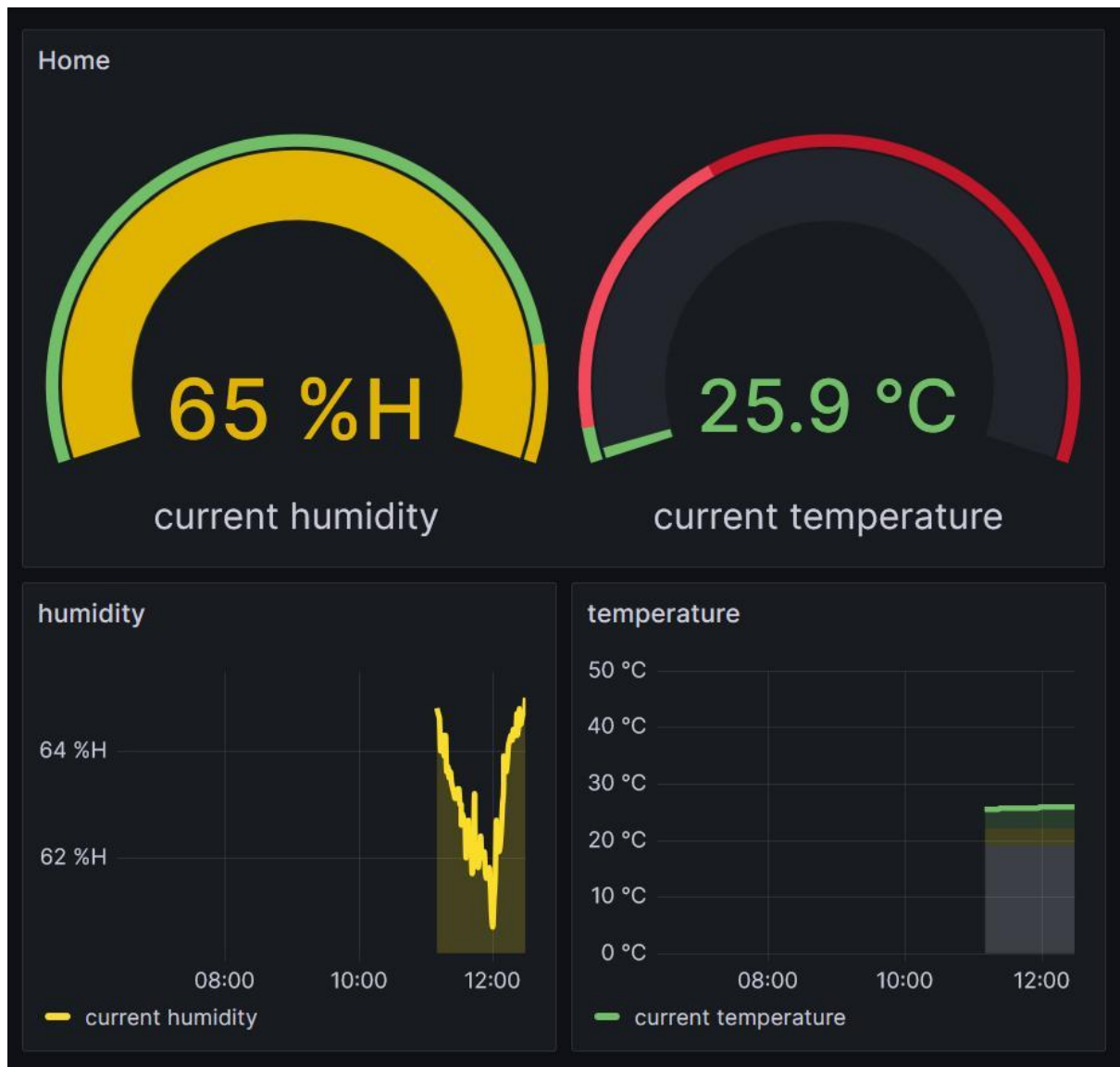


Figure 3: Mobile Monitoring Dashboard



9. Sample Output

Temperature: 29°C

Humidity: 68%

Wi-Fi Status: Connected

Cloud Status: Active

10. Applications

- Smart homes
- Weather monitoring
- Greenhouses
- Agriculture
- Industrial monitoring
- Server room monitoring

11. Advantages

- Real-time monitoring
- Remote access through IoT
- Low power consumption
- Easy implementation
- Low-cost system
- Accurate environmental sensing

12. Conclusion

The Temperature & Humidity Monitoring System is a simple and effective IoT project using Arduino or Raspberry Pi. It demonstrates real-time environmental monitoring and cloud connectivity. The project is cost-effective, easy to build, and suitable for students learning IoT and embedded systems.

MOTION DETECTION ALARM SYSTEM

1. Introduction

The Motion Detection Alarm System is a simple IoT-based security project that detects human movement using a PIR (Passive Infrared) sensor. When motion is detected, the system activates a buzzer/alarm and can send alerts through Wi-Fi or cloud applications.

This project is useful for home security, office monitoring, smart surveillance, and intrusion detection systems.

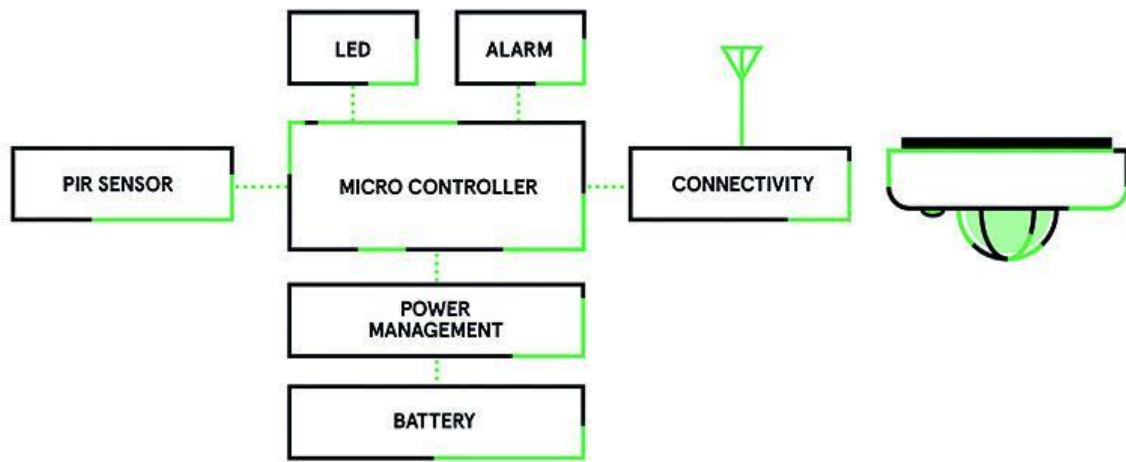
2. Components Required

Component	Quantity
Arduino Uno / Raspberry Pi	1
PIR Motion Sensor	1
Buzzer	1
LED	1
ESP8266 Wi-Fi Module	1
Breadboard	1
Jumper Wires	Several
Power Supply	1

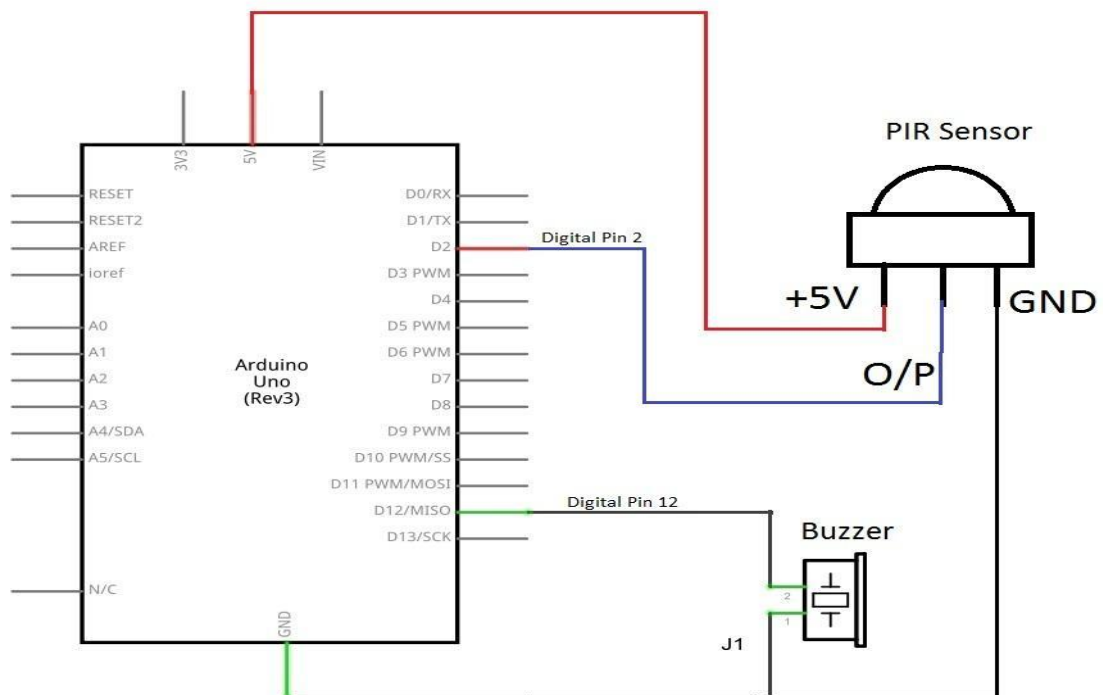
3. Working Principle

1. PIR sensor detects motion or body heat.
2. Arduino/Raspberry Pi reads sensor data.
3. If motion is detected:
 - LED turns ON
 - Buzzer alarm activates
4. ESP8266 sends notification to mobile/cloud platform.
5. System continues monitoring for further movement.

4. System Architecture Figure



5. Circuit Diagram Figure



6. Arduino Code

```
int pirSensor = 2;

int buzzer = 8;

int led = 13;

void setup() {
  pinMode(pirSensor, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(led, OUTPUT);

  Serial.begin(9600);
  Serial.println("Motion Detection Alarm System");
}

void loop() {
  int motion = digitalRead(pirSensor);
  if (motion == HIGH)
    digitalWrite(led, HIGH);
    digitalWrite(buzzer, HIGH);
    Serial.println("Motion Detected!");
    delay(3000);

  } else {

    digitalWrite(led, LOW);
    digitalWrite(buzzer, LOW);
    Serial.println("No Motion");
    delay(1000);
  }
}
```

7. Raspberry Pi Python Code

```
import RPi.GPIO as GPIO
import time

PIR = 17
BUZZER = 18
LED = 23

GPIO.setmode(GPIO.BCM)

GPIO.setup(PIR, GPIO.IN)
GPIO.setup(BUZZER, GPIO.OUT)
GPIO.setup(LED, GPIO.OUT)

print("Motion Detection Alarm System")

try:
    while True:

        motion = GPIO.input(PIR)

        if motion:

            GPIO.output(BUZZER, True)
            GPIO.output(LED, True)

            print("Motion Detected!")
```

```
time.sleep(3)
```

```
else:
```

```
GPIO.output(BUZZER, False)
```

```
GPIO.output(LED, False)
```

```
print("No Motion")
```

```
time.sleep(1)
```

```
except KeyboardInterrupt:
```

```
GPIO.cleanup()
```

8. Output Images

Figure 1: Motion Detection Hardware Prototype

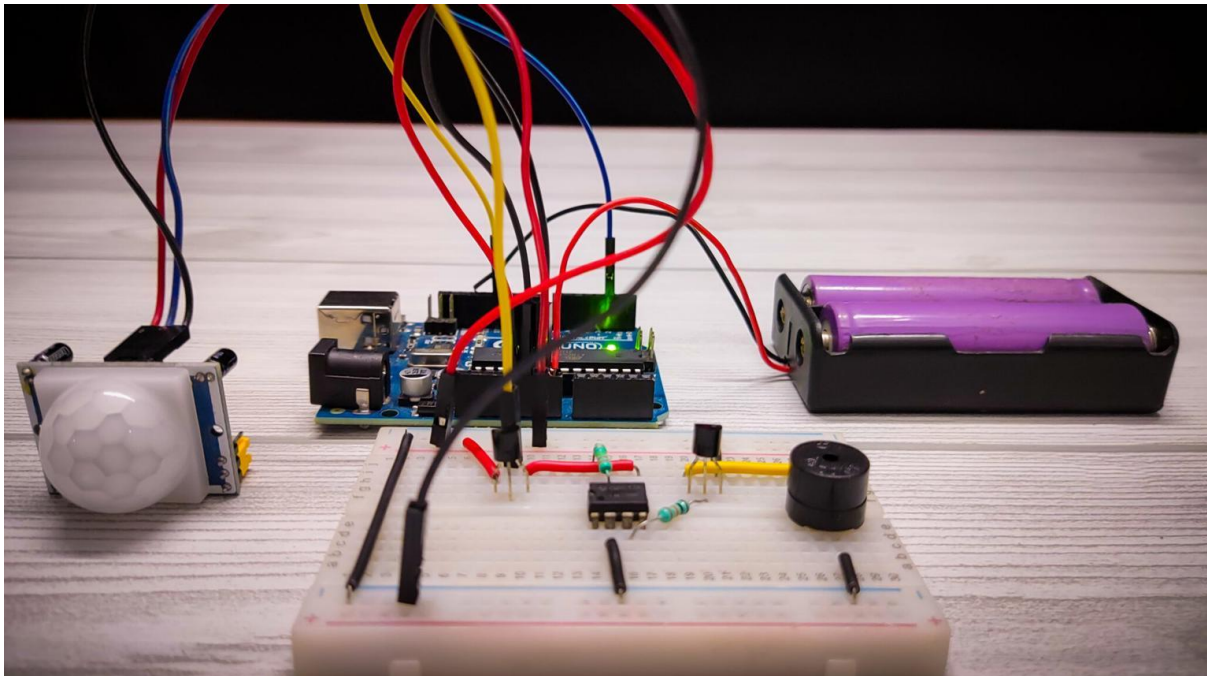


Figure 2: Motion Detected Output

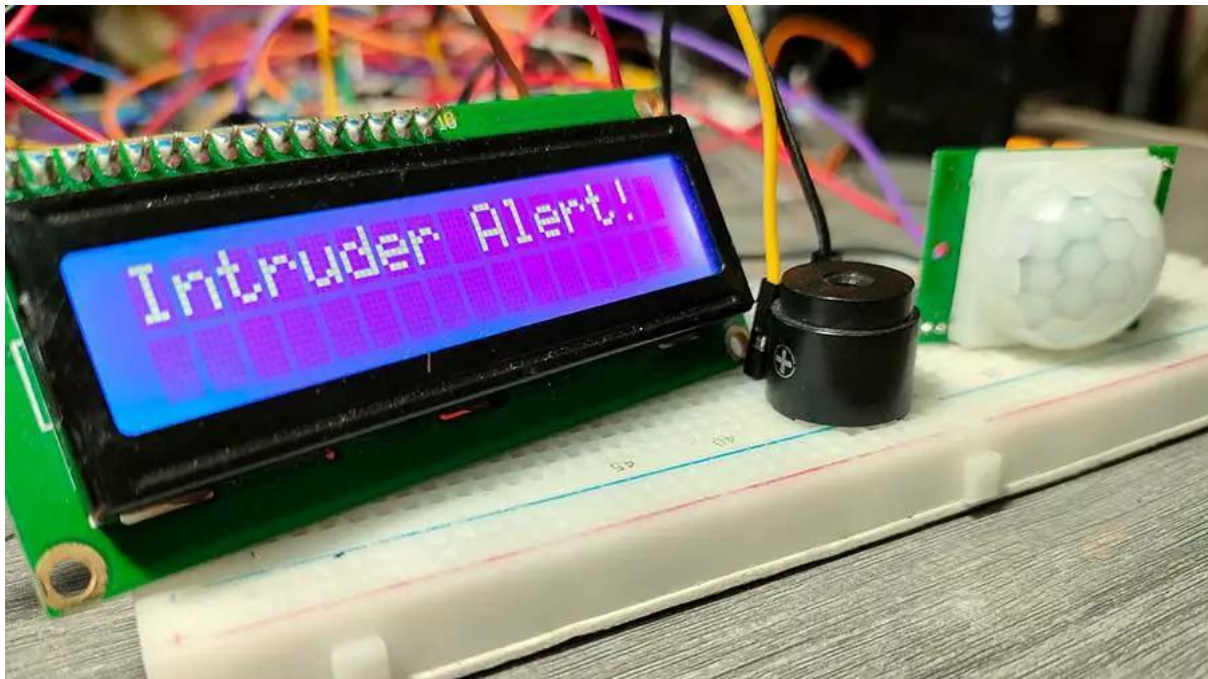
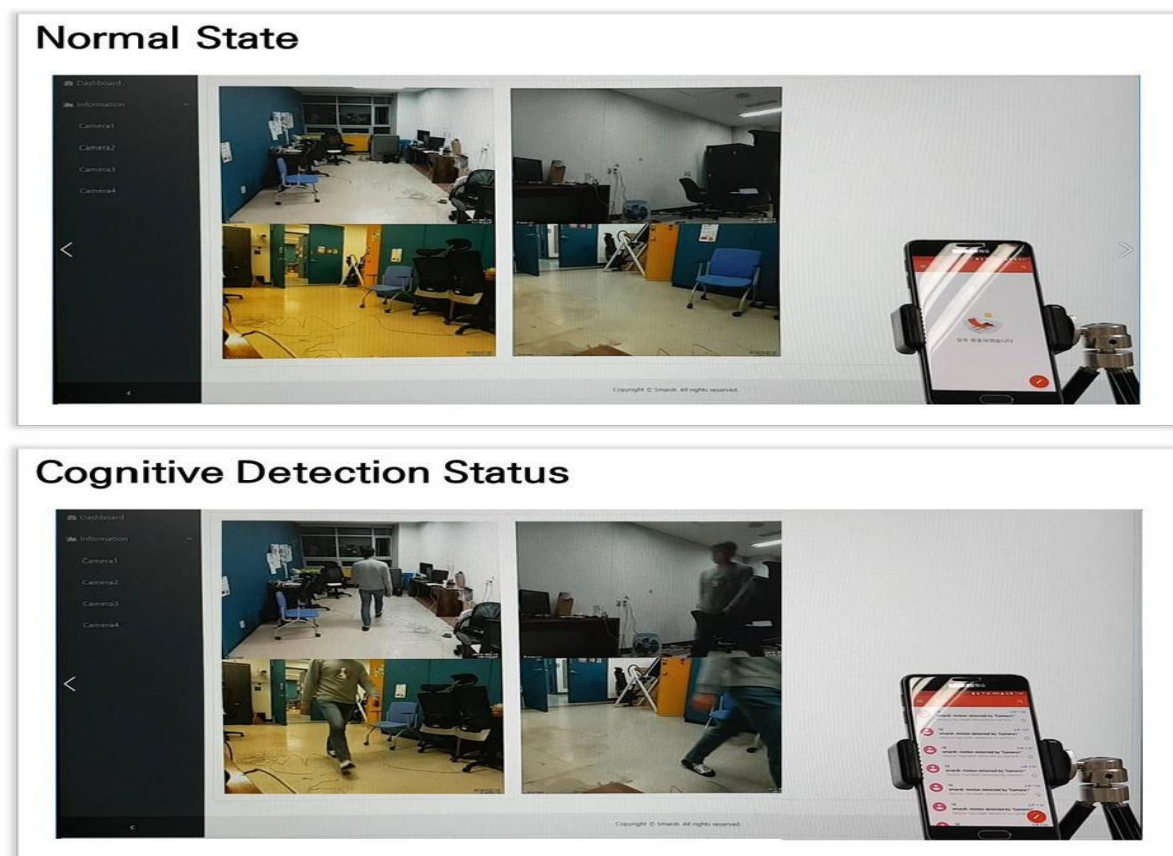


Figure 3: Mobile Alert Monitoring



9. Sample Output

Motion Detection Alarm System

Motion Detected!

Buzzer Activated

Security Alert Sent

No Motion

System Monitoring...

10. Applications

- Home security systems
- Office surveillance
- Smart buildings
- Intrusion detection
- Industrial monitoring
- Smart IoT security systems

11. Advantages

- Real-time motion detection
- Simple and low-cost implementation
- Remote IoT monitoring
- Easy to install
- Low power consumption
- Improved security

12. Conclusion

The Motion Detection Alarm System is a simple and effective IoT-based project using Arduino or Raspberry Pi. It provides real-time motion detection and alarm functionality for security applications. The project is easy to implement and suitable for students learning IoT, sensors, and embedded systems.